Adaptive Retrieval-Augmented Generation

Wenlong Zheng
Department of Electrical and Computer Engineering
University of Toronto
wenlong.zheng@mail.utoronto.ca

Axel Tang
Department of Electrical and Computer Engineering
University of Toronto
axel.tang@mail.utoronto.ca

Tanish Upreti
Department of Electrical and Computer Engineering
University of Toronto
tanish.upreti@mail.utoronto.ca

Zihan Gong
Department of Electrical and Computer Engineering
University of Toronto
zihan.gong@mail.utoronto.ca

Abstract

Retrieval-Augmented Generation (RAG) has been a significant advancement in the field of natural language processing, enabling Large Language Models (LLMs) to generate more accurate and contextually relevant responses by leveraging external knowledge sources. In this project, we explore the implementation of an efficient and effective Adaptive RAG system that routes the query to bypass retrieval or engage in iterative search from Wikipedia corpus to enhance the quality of generated text in Question-Answering (QA) task. We detail our design choices, methodologies, and the results of our experiments, demonstrating the effectiveness of our approach in various benchmark such as TriviaQA, and Natural Questions.

Assentation of Teamwork

Axel Tang, Wenlong Zheng, Tanish Upreti and Zihan Gong have equally contributed to the project. Here is the breakdown of each member's contribution:

Individual Contributions

Axel Tang: Implementation of the Retrieval module and data preprocessing.
Wenlong Zheng: Design and implementation of the Generation Module, Adaptive Router Module, and Trainer scripts.
Tanish Upreti: Integration of Retrieval and Generation modules, and experimentation.
Zihan Gong: Evaluation metrics design and result analysis.

ECE 1508: Applied Deep Learning (Fall 2025)

1 Introduction

Large Language Models (LLMs) has proven their capabilities to process natural language tasks, generating coherent and human-like responses. The rapid advancement in this field has led researchers and engineers are actively exploring the potential of LLM centered systems like agents and more refined methodologies for factual grounding.

Retrieval Augmented Generation (RAG)[8], is a pivotal technique utilized to enhance accuracy of AI generated text by coupling question with relevant information retrieved from documents or databases. While standard RAG provides a strong baseline, it also introduces computational overhead, particularly for effortless questions that an LLM can accurately respond solely. This project summarizes a baseline implementation of this methodology and the design and effectiveness of an Adaptive RAG System. We focus on dynamically optimizing the retrieval process to balance computational overhead with enhanced performance on complex Question Answering (QA) tasks.

2 Preliminaries and Problem Formulation

2.1 Preliminaries

We begin by introducing the preliminaries of different strategies of Retrieval Augmented Generation.

Naive RAG To eliminate the limitations of LLM's internal knowledge, Single-Step Retrieval (Naive RAG)[8] has been introduced by attaching an external memory source to the LLM. In this approach, a retriever R first selects top-k relevant documents d from corpus C based on query X. The LLM will take both the query and retrieved documents as input to produce answer Y, formally defined as follows: $Y = LLM(X, d)$. This process allows LLM to ground its answer in factual documents, and it can subsequently improve the accuracy and concurrency of LLMs for QA. This approach works well for a simple queries that answer is explicitly contained in single document or a small set of documents.

Iterative RAG The Naive RAG[8] is prone to complex, multi-hop questions as the answer is distributed across different documents that do not share significant lexical overlap with the original query. Iterative RAG[11] approach aims to solve this problem by allowing LLM to iteratively interact with corpus C to collect all necessary documents required to solve advanced, multi-hop questions. In Iterative RAG, the model generates partial answer, which serve as query for the next retrieval process. Let d_t denote the documents retrieved at step t. The generation at step t is conditioned on the original query X and the cumulative context retrieved so far $D = d_1, \ldots, d_t$. This adaptive process continues until the model gathers sufficient information to form the final answer Y, modeled as formula 1

$$LLM(Y|X) = \sum_{Z} LLM(Y|X, D)R(D|X) \tag{1}$$

This approach is powerful, but it can result in vast expansion of computation overhead as multiple access to retriever and LLM required.

2.2 Problem Formulation

While the Iterative RAG[11] successfully handle advanced questions, a critical challenge arises from the computational overhead and increased latency when applying iterative retrieval mechanisms to simple, readily solvable queries by non-retrieval based approach.

We have tested Qwen 2.5 7B Instruct model[10] on 1000 samples from TriviaQA[5], and Natural Questions[7] benchmark each with non-retrieval approach, and Naive RAG[8] by Dense Passage Retrieval (DPR)[6] with exact match as the evaluation metrics shown in Table 1. The results indicate that while retrieval-augmented methods significantly enhance performance on complex queries, a substantial portion of simple queries can be accurately addressed without retrieval, highlighting the inefficiency of blanket retrieval strategies.

Therefore, we break down our objective into two parts: We aim to optimize the objective of maximize the probability of the correct answer y with given query x and corpus C. The second part is to achieve this objective efficiently by dynamically determining whether to retrieve from C each step and how to integrate the retrieved documents D to generate accurate and factual-based responses.

Table 1: LLM Performance (Exact Match)

Benchmark	Model	Retrieval	Non-Retrieval	Naive RAG
TriviaQA	Qwen 2.5 7B Instruct	DPR	54.1%	64.0%
Natural Questions	Qwen 2.5 7B Instruct	DPR	21.8%	44.9%

3 Solution via Deep Learning

In this section, we demonstrate the overall architecture, key component design choices, and implementation specifics of our Adaptive Retrieval-Augmented Generation (Adaptive RAG) system. We segment our system into three primary components—the Adaptive Routing Module, the Retrieval Module, and the Generation Module, each composed of specialized Deep Learning model to address specific tasks within the overall framework.

3.1 Retrieval Module

Retrieval Module is responsible for retrieving top-k relevant documents for input query q from corpus c. Retrieval Module is composed of chunking, embedding, indexing, and searching.

Chunking Corpus are segmented into chunks that act as the atomic units of knowledge

Embedding All chunks were transformed into high-dimensional dense vectors using a pre-trained embedding model

Indexing Faiss[2] is utilized to construct search index of dense vectors to enable low-latency similarity search between query and corpus.

Searching The input query q is embedded into vectors using the same embedding model used for constructing the corpus embeddings, and similarity search is performed by Indexing to retrieve top-k relevant documents.

3.2 Adaptive Routing Module

Adaptive Routing Module processes input query q with a pre-trained LLM. The output of this module is either to search for relevant documents from corpus c by breaking down the query into sub-queries, or to terminate retrieval process to generate final answer from LLM. In Adaptive Routing Module, the model generates multiple sub queries related to original query, which serve as queries for the next retrieval process. Let d_t denote the documents retrieved at step t. The generation at step t is conditioned on the original query X and the cumulative context retrieved so far $D = d_1, \ldots, d_t$. This adaptive process continues until the model gathers sufficient information to form the final answer Y

3.3 Generation Module

Generation Module is tasked to construct final answer for the input query q by pretrained LLM with cumulative documents retrieved from Adaptive Routing Module.

4 Implementation

The implementation details of Retrieval Module and Adaptive Routing Module are described in the following subsections.

4.1 Retrieval Module Implementation

We implement the Retrieval Module by constructing dense vector index from Wikipedia corpus with Faiss[2] to enable efficient similarity search using BGE-M3[1] embedding model. The Faiss index is built with IVF65536_PQ128 index type to optimize memory efficiency, and original documents are stored in sqlite database. During the retrieval process, the input query is embedded using the same BGE-M3[1] model, and top-k relevant document ids are retrieved by Faiss[2], used to fetch the corresponding documents from the sqlite database.

Table 2: Retrieval Performance (Recall@20)

Model	TriviaQA	Natural Questions	Memory Usage
DPR	83.4%	77.3%	60GB
BM25	90.3%	62.4%	4GB
BGE-M3	90.9%	69.2%	10GB

4.2 Adaptive Routing Module Implementation

The Adaptive Routing Module is designed to dynamically decide whether to retrieve documents or generate the final answer based on the input query and the context accumulated from previous retrievals each step. The module is built upon the Qwen 2.5 7B Instruct model[10], which is fine-tuned by Group Relative Policy Optimization (GRPO)[12] algorithm by LoRA[4] to learn optimal policy within the group that maximizes accuracy of final answer with minimal retrieval complexity.

Fine-tuning Data The fine-tuning data consists of 1000 samples, with 400 samples from TriviaQA[5] and 600 samples from Natural Questions[7] benchmark. Each sample includes a query, the corresponding answer. You may refer to Appendix A for more details about the fine-tuning implementation.

5 Numerical Experiments

5.1 Retrieval Module Experiments

The retrieval model for Retrieval Module is evaluated on both dense and sparse retrieval methods with Wikipedia corpus on 1000 samples from both TriviaQA[5] and Natural Questions[7] benchmarks by top 20 retrievals for each question. Our evaluation specifically assesses the trade-off between retrieval performance and memory usage. Three retrieval methods are implemented and compared: Dense Passage Retrieval (DPR)[6], BM25, and our approach with BGE-M3[1]. The implementation of DPR and BM25 are based on existing libraries, pyserini[9]. The performance of each retrieval method is summarized in Table 2. As observed, our constructed retrieval model using BGE-M3[1] achieves comparable retrieval performance to DPR[6] while significantly reducing memory consumption by approximately 80%.

5.2 RAG System Experiments

5.2.1 Experiment Setup

All RAG systems are evaluated on 1000 samples from both TriviaQA[5] and Natural Questions[7] benchmarks with Qwen 2.5 7B Instruct model[10] as the base LLM, and BGE-M3[1] as retrieval model on Wikipedia corpuss with max of 10 steps. The RAG systems compared include Non-Retrieval, Naive RAG[8], Iterative RAG, and our Adaptive RAG system. The evaluation metric used is Exact Match (EM) score, which measures the percentage of final answers that match any one of the ground truth answers exactly, average number of steps conducted by Router Module, average number of documents retrieved from Retrieval Module, and total time spent.

5.3 Results

Table 3 represents the performance comparison of different RAG systems on TriviaQA[5] and Natural Questions[7] benchmarks. The results indicate that our Adaptive RAG system achieves a significant improvement over Non-Retrieval approach, with an EM score increase of 22.5% on TriviaQA[5] and 16.8% on Natural Questions[7]. Compared to Iterative RAG, our Adaptive RAG system demonstrates competitive performance while substantially reducing the average number of retrieval steps and documents accessed, highlighting its efficiency in balancing accuracy and computational overhead. As comparison to original Qwen 2.5 7B Instruct model[10], the fine-tuned model uses fewer steps on the easier TriviaQA benchmark[5] with same accuracy, but automatically expands its search for the more difficult questions in Natural Questions[7] as desired by our Adaptive RAG system.

Table 3: Comparison of RAG Systems on TriviaQA and NQ Benchmarks

Router Model	RAG System	EM	Avg. Step	Avg. Doc
TriviaQA_1000				
None	None	54.1%	-	-
Qwen-2.5-7B-Instruct	Native	74.1%	1	1
Qwen-2.5-7B-Instruct	Iterative	**79.4%**	10	4.84
Qwen-2.5-7B-Instruct	Adaptive	76.6%	1.031	1.94
Qwen-2.5-7B-Instruct-Finetuned	Adaptive	76.6%	1.024	1.95
Natural Questions_1000				
None	None	21.8%	-	-
Qwen-2.5-7B-Instruct	Native	37.1%	1	1
Qwen-2.5-7B-Instruct	Iterative	**43.9%**	10	5.00
Qwen-2.5-7B-Instruct	Adaptive	38.4%	1.032	1.75
Qwen-2.5-7B-Instruct-Finetuned	Adaptive	38.6%	1.053	1.77

6 Discussion

The Adaptive RAG system demonstrates an approach to balancing accuracy and efficiency in retrieval-augmented generation tasks. By adaptively determining retrieval strategies based on the complexity of the query, the system effectively reduces unnecessary computational overhead while maintaining competitive performance. The fine-tuning of the Adaptive Routing Module using GRPO[12] further enhances its decision-making capabilities, allowing it to optimize retrieval strategies based on the specific requirements of each query.

However, we only have minor improvements from fine-tuning. The possible reason is that the base model Qwen 2.5 7B Instruct[10] already has strong capabilities in understanding and generating text, limiting the potential gains from further fine-tuning on a relatively small training dataset within a short range of epochs. From a different perspective, the fine-tuning dataset from current benchmarks may not be sufficiently diverse or large enough to provide significant features in routing decisions for the model to learn from, resulting in marginal improvements. Furthermore, the defined reward structure can be trapped in a suboptimal policy for the routing decisions required for this specific task.

There are areas for further improvement on fine-tuning. The current training implementation could benefit from more extensive labeled and better structured fine-tuning data to better capture the nuances of various question types through additional epochs. Additionally, exploring alternative reward structures in the training process may yield further enhancements in routing decisions. Future work could also investigate the integration of more advanced retrieval techniques to further improve the system's performance across a wider range of queries.

7 Conclusions

In this project, we have developed an Adaptive Retrieval-Augmented Generation (Adaptive RAG) system that effectively balances the trade-off between accuracy and computational efficiency in question-answering tasks. By incorporating an Adaptive Routing Module that adaptively selects retrieval strategies, our system demonstrates better performance over traditional RAG system with reasonable computational cost. The use of GRPO[12] for fine-tuning the routing policy has proven effective in optimizing retrieval strategies. Our experiments on benchmarks such as TriviaQA[5] and Natural Questions[7] validate the efficacy of our approach. Future work will focus on expanding the fine-tuning dataset, refining the reward structure, and exploring performance on multi-hop QA benchmarks such as MuSiQue[13], and 2WikiMultihopQA[3].

References

[1] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2023.

[2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.

[3] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In Donia Scott, Nuria Bel, and Chengqing Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.580. URL https://aclanthology.org/2020.coling-main.580/.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

[5] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension, 2017. URL https://arxiv.org/abs/1705.03551.

[6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.550. URL https://www.aclweb.org/anthology/2020.emnlp-main.550.

[7] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: a benchmark for question answering research. *Transactions of the Association of Computational Linguistics*, 2019.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *CoRR*, abs/2005.11401, 2020. URL https://arxiv.org/abs/2005.11401.

[9] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations. In *Proceedings of the 44th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2021)*, pages 2356–2362, 2021.

[10] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

[11] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9248–9274, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.620. URL https://aclanthology.org/2023.findings-emnlp.620/.

[12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

[13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition, 2022. URL https://arxiv.org/abs/2108.00573.

A GRPO Training Implementation Details

Fine-tuning by Chunked Group Relative Policy Optimization The fine-tuning process involves optimizing the routing policy from multi-step rollout each consists of multiple steps where at each step, the model decides whether to retrieve documents by generating subtopics or generate the final answer with cumulative conversation history, requiring a large computation overhead. To address this challenge, we segment the rollout history inot individual steps containing rollout history up to current step only, and each step is treated as an independent sample for policy optimization, processed by chunks to fit into GPU memory.

Reward Design The reward signal is based on the accuracy of the final answer, response format, relevance between original question and sub-questions, and retrieval efficiency. A positive reward is given for correct answers with right format and relevant sub-queries, while penalties are applied for each extra step taken, wrong format, irrelevant sub-queries, and incorrect answers.

Fine-tuning Parameters The Adaptive Routing Module is implemented by fine-tuning Qwen 2.5 7B Instruct model[10] with LoRA[4] on our preprocessed dataset. The training is conducted on a 4 NVIDIA H800 GPU with 80GB memory, using a group size of 8, chunk size of 3, batch size of 2, and a learning rate of 8e-8 over 3 epochs.

Contest for Information Sharing

As part of this course, we may share selected project materials (e.g., reports, presentation slides, and presentation recordings) on the course webpage as learning resources for future students. Additionally, we may use anonymized project information for internal statistical analysis of course outcomes. Please indicate your preferences below. *Your choices will not affect your grade in any way.*

Consent for Sharing Project Materials

- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Optional Comments

Please list any specific conditions or comments here.

Group Identification

- Group number: 1
- Names of group members: Wenlong Zheng, Axel Tang, Tanish Upreti, Zihan Gong
- Signature of of group members: Wenlong Zheng, Axel Tang, Tanish Upreti, Zihan Gong
- Date: 2025-12-11